배열

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Java
C++

배열

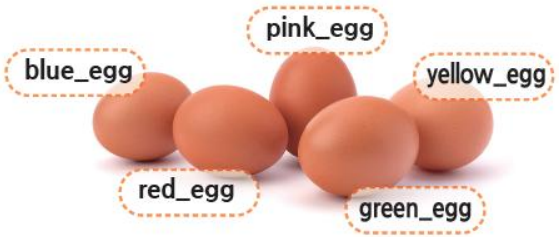
변수와 배열의 차이

데이터를 저장할 때 사용하는 방법은 변수를 이용해서 데이터를 저장

변수	배열
하나의 데이터를 저장	많은 변수를 다루어야 할 때 보다 효과적인 방법으로 다룰 수 있도록 데이터 구조를 제공

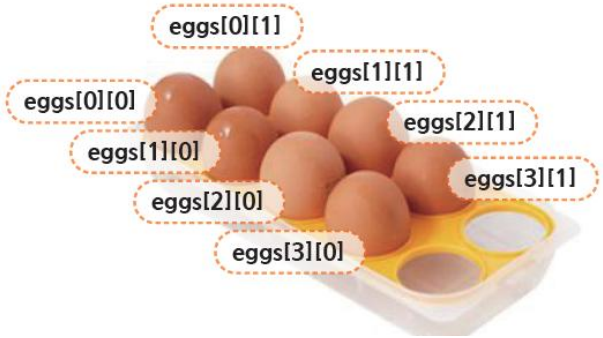
배열

변수와 배열의 차이




```
fried_egg(blue_egg);
fried_egg(red_egg);
fried_egg(green_egg);
fried_egg(pink_egg);
fried_egg(yellow_egg);
```

모든 계란을 각각 프라이 해야 한다.
계란이 증가하면 프로그램이 계속 추가 되어야 한다.




```
for ( j = 0; j <= 3; j++)
  for (k =0 ; k <= 1' k++)
    fried_egg(eggs[j][k]);
```

계란 판의 계란을 순환하면서 프라이 한다.
계란이 증가해도 프로그램이 추가되지 않는다.

배열

배열은 동일한 자료형으로 선언된 데이터 공간을 메모리상에 연속적으로 나열하여
데이터 관리를 편하게 할 수 있도록 해 줌

배열도 인스턴스로 다룸

인스턴스와 같이 배열(묶음)의 메모리 주소 값을 배열 변수에 저장해서 배열의 요소에 접근

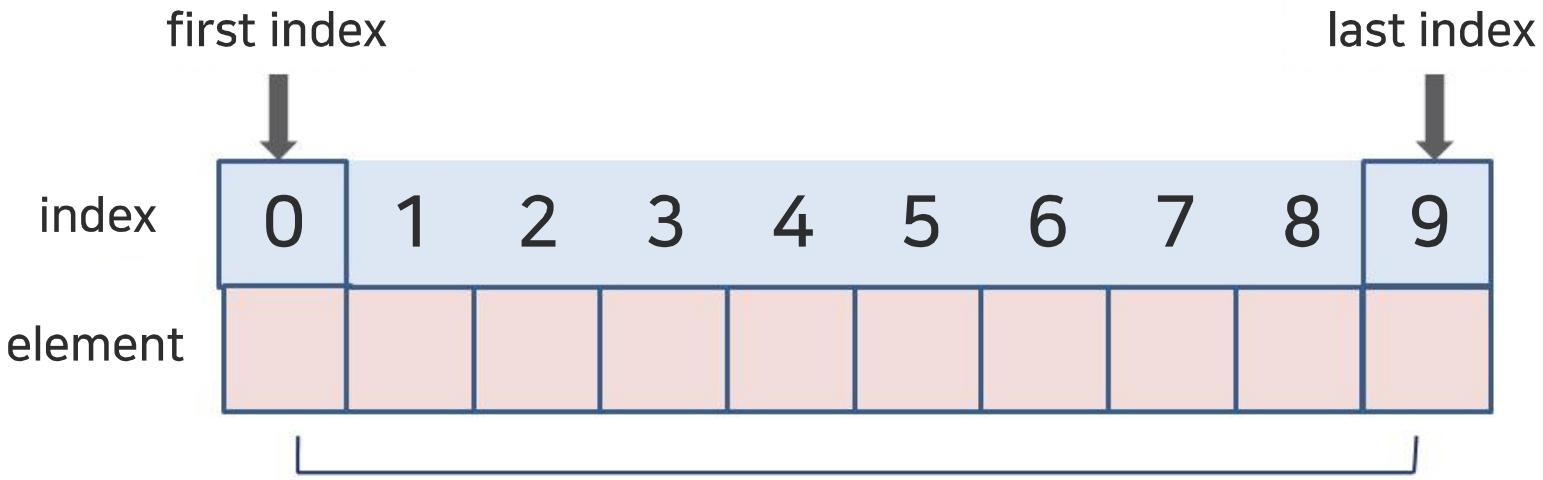
배열은 1차원 배열과 다차원 배열로 정의해서 사용

배열

배열은 같은 타입의 데이터를 연속된 공간에 나열 시키고 각 데이터에 인덱스(index)를 부여해 놓은 자료구조

배열의 구성은 각각의 값을 구성하는 배열의 요소와 배열의 요소 위치를 정의하는 인덱스로 구성

배열의 요소는 각각의 변수이고, 인덱스는 0부터 시작하는 숫자로 양수의 값으로만 표현함



배열 요소의 개수: 10

1차원 배열의 선언과 생성

배열 변수 선언

배열 변수 선언 시에 정의하는 자료형 타입은 배열 요소로 저장되는 변수의 타입을 명시함

배열 이름 변수에는 배열 인스턴스의 메모리 주소를 저장함

배열 변수는 배열이 생성된 후에 배열 인스턴스에 접근하기 위해 사용함

배열 변수 선언 방법	자료형타입 [] 배열이름 자료형타입 배열이름 []

배열의 생성

배열을 생성할 때는 클래스의 인스턴스를 생성할 때처럼 new 키워드를 이용하여 배열의 인스턴스를 생성함

[] 괄호 안에 생성하고자 하는 배열의 요소의 개수를 지정함

배열의 요소 개수는 0보다 큰 정수 값을 기술해주고 자료형 타입은 배열 변수 선언 시에 정의한 자료형타입과 같은 타입을 기술해 주어야 함

배열 생성 방법

배열 이름 = new 자료형타입 [배열 요소의 개수]

```
int[] numbers; // 정수 타입의 배열 변수 선언
numbers = new int[5]; // 정수형 변수 5개를 요소로 가지는 배열 인스턴스 생성
```

배열의 생성

배열 생성 과정

int[] numbers ;

numbers

배열 인스턴스의 주소 값을 저장할 배열 변수 number 생성

numbers = new int[5];

numbers

0x100

int 타입의 변수 5개를 저장할 수 있는 배열 인스턴스를 생성하고 인스턴스의 주소 값을 numbers 변수에 저장
numbers = 0x100;

0x100

int 변수	int 변수	int 변수	int 변수	int 변수
0	0	0	0	0

배열의 생성

배열 변수 선언과 배열 인스턴스 생성의 예

int[] nums = new int[10]; int 타입의 요소를 10개 저장할 수 있는 배열 생성

byte[] nums = new byte[5]; byte 타입의 요소를 5개 저장할 수 있는 배열 생성

long[] nums = new long[12]; long 타입의 요소를 12개 저장할 수 있는 배열 생성

boolean[] checks = new boolean[5] boolean 타입의 요소를 5개 저장할 수 있는 배열 생성

char[] chars = new char[3]; char 타입의 요소를 3개 저장할 수 있는 배열 생성

String[] names = new String[10]; 문자열 요소를 10개 저장할 수 있는 배열 생성

배열의 요소 접근 방법

배열 인스턴스의 주소 값을 가지는 배열 변수를 통해 접근

배열의 요소는 순서를 가지고 있고 이 순서를 통해 순차적 접근이 가능

반복문을 이용해서 배열의 요소에 참조 또는 일괄 처리가 가능

배열의 요소에 접근하는 방법은 배열의 이름과 인덱스 값으로 배열의 요소에 접근

numbers | 0x100

numbers [0]	numbers [1]	numbers [2]	numbers [3]	numbers [4]
0	0	0	0	0

numbers [index] → numbers [0] = 100;

배열의 요소 접근 방법

배열의 요소에 접근하는 방법은 배열 변수의 이름과 [] 괄호를 사용하고
괄호 안에 인덱스 값을 명시하여 배열의 요소를 참조함

인덱스 값은 배열의 요소 개수가 N개일 때 index 값은 0부터 N-1까지임

```
System.out.println( numbers[0] );
```

1차원 배열도 인스턴스인데 이 배열 인스턴스에는 1차원 배열의 요소 개수를 저장하고 있는
length 변수가 있음

length 변수를 사용하는 반복문을 이용하면 공통 작업처리를 간단하게 처리할 수 있음

```
int[] arr = new int[5];
arr.length; // 1차원 배열의 개수 : 5
```

배열의 요소 접근 방법

```
package chapter06;
public class ArrayTest1 {
        public static void main(String[] args) {
                // 배열 인스턴스 생성
                int[] korScores = new int[10];
                // 배열 요소 초기
                korScores[0]=68;
                korScores[1]=65;
                korScores[2]=86;
                korScores[3]=91;
                korScores[4]=65;
                korScores[5]=78;
                korScores[6]=96;
                korScores[7]=76;
```

배열의 요소 접근 방법

```
korScores[8]=82;
korScores[9]=75;
int sum=0; // 총점
float avg=0f; // 평균
// 배열은 반복문을 통해 일괄 처리가 가능합니다.

for(int i=0; i<10; i++){
        sum +=korScores[i];        // 배열의 요소들의 합을 구합니다.
}
System.out.println("국어 점수의 총합은 "+ sum + "점 입니다.");
System.out.println("국어 점수의 평균 점수는 "+ sum/korScores.length +
"점 입니다.");

    }

}
```

배열의 선언과 초기화

변수의 선언하는 방법 중에 선언과 초기화를 한 번에 처리할 수 있음

배열의 선언과 동시에 배열의 요소를 바로 초기화 함

```
자료형 타입[]변수이름 = {배열의 요소, 배열의 요소, 배열의 요소...};
자료형 타입[] 변수이름 = new 자료형타입[]{배열의 요소, 배열의 요소, ...};
```

```
int[] score = {100, 90, 70, 80, 65};

int[] score;
score = new int[]{100, 90, 70, 80, 65};

int[] score = new int[]{100, 90, 70, 80, 65};
```

배열의 선언과 초기화

```
package chapter06;

public class ArrayTest2 {

	public static void main(String[] args) {

		// 배열 인스턴스 생성
		int[] score1 = { 100, 90, 70, 80, 65 };
		int[] score2 = new int[] { 100, 90, 70, 80, 65 };
		int[] score3;
		score3 = new int[] { 100, 90, 70, 80, 65 };

		System.out.println("배열 score1 =====");
		for (int i = 0; i < score1.length; i++) {
			System.out.print(score1[i] + ", ");
		}
```

배열의 선언과 초기화

```
            System.out.println();
            System.out.println("배열 score2 =====");
            for (int i = 0; i < score2.length; i++) {
                System.out.print(score2[i] + ", ");
            }

            System.out.println();
            System.out.println("배열 score3 =====");
            for (int i = 0; i < score3.length; i++) {
                System.out.print(score3[i] + ", ");
            }
        }

}
```

매개변수의 인자에 배열 전달

메소드의 매개변수를 참조변수로 선언하고 참조 값을 인자로 전달 가능

배열 변수도 매개변수로 정의해서 사용 가능

매개변수에 배열 변수로 선언된 경우 메소드 호출 시 매개변수의 인자로 배열 인스턴스의 참조 값을 전달

매개변수의 인자에 배열 전달

```
package chapter06;
public class ArrayTest3 {
        public static void main(String[] args) {
                // 배열 인스턴스 생성
                int[] korScores = new int[10];
                // 배열 요소 값 대입
                korScores[0]=68;
                korScores[1]=65;
                korScores[2]=86;
                korScores[3]=91;
                korScores[4]=65;
                korScores[5]=78;
                korScores[6]=96;
                korScores[7]=76;
```

매개변수의 인자에 배열 전달

```
                korScores[8]=82;
                korScores[9]=75;
// 합과 평균을 계산하고 출력하는 메서드 호출
                sum(korScores);  //메소드 호출 시 배열 전달

        }

        public static void sum(int[] arr) { //배열의 주소 값을 전달 받음

                int sum = 0;
                float avg = 0f;

                // 배열은 반복문을 통해 일괄 처리가 가능합니다.
                for (int i = 0; i < 10; i++) {
                        sum += arr[i]; // 배열의 요소들의 합을 구합니다.
                }
```

매개변수의 인자에 배열 전달

```
                System.out.println("국어 점수의 총합은 " + sum + "점 입니다.");
                System.out.println("국어 점수의 평균 점수는 " + sum / arr.length + "점 입니다.");

        }

}
```

다차원배열 (2차원 배열)

COMPILER DESIGN
Reinhard Wilhelm • Dieter Maurer
COMPILER DESIGN
Java
C++

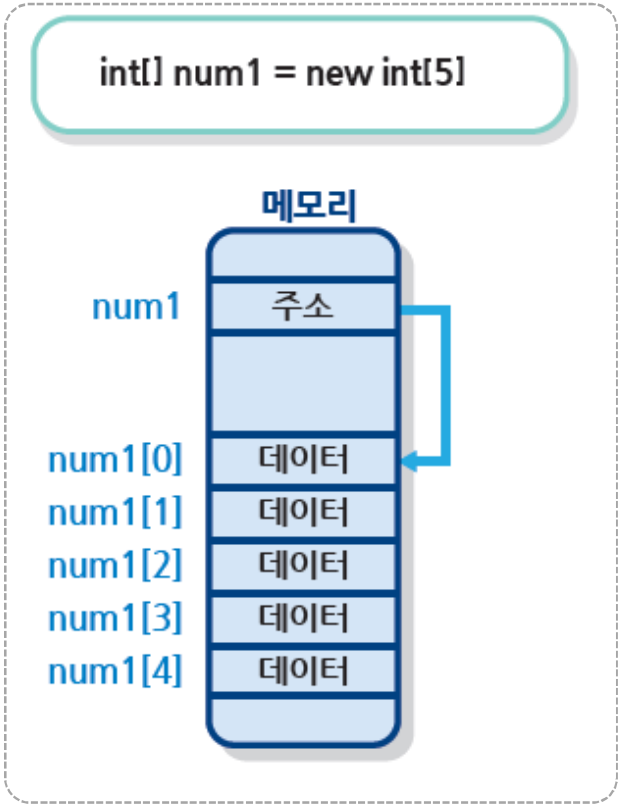
다차원배열(2차원 배열)

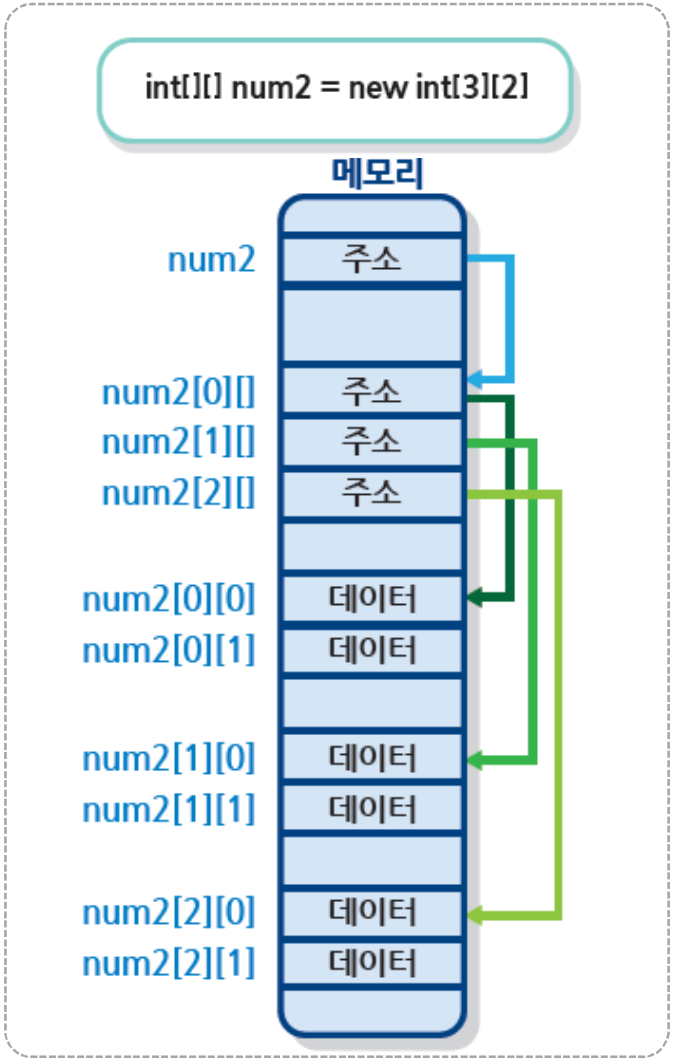
다차원배열이란 2차원 이상의 배열을 의미하며, 배열의 요소가 다른 배열을 가지는 배열을 의미함

2차원 배열은 1차원 배열을 요소로 가지는 1차원 배열을 의미함

배열의 각 요소가 1차원 배열 인스턴스의 주소 값을 저장하고 있음

다차원배열(2차원 배열)


int[] num1 = new int[5]
메모리
num1
주소
num1[0]
num1[1]
num1[2]
num1[3]
num1[4]
데이터
데이터
데이터
데이터
데이터


int[][] num2 = new int[3][2]
메모리
num2
주소
num2[0][]
num2[1][]
num2[2][]
주소
주소
주소
num2[0][0]
num2[0][1]
num2[1][0]
num2[1][1]
num2[2][0]
num2[2][1]
데이터
데이터
데이터
데이터
데이터
데이터

2차원 배열의 선언과 생성

```
자료형타입[][] 배열변수이름;
자료형타입 배열변수이름[][];
자료형타입[] 배열변수이름[];
```

2차원 배열은 행과 열을 사용해서 표(table)처럼 표현해볼 수 있음

➜ 첫 번째 []는 행을 정의하고 두 번째 []는 열을 정의

```
int[][] numbers = new int[3][3];
```

numbers[0][0]	numbers[0][1]	numbers[0][2]
numbers [1][0]	numbers [1][1]	numbers [1] [2]
numbers [2][0]	numbers [2] [1]	numbers [2][2]

2차원 배열의 접근

배열의 각 요소에 접근하는 방법은 행의 index 값과 열의 index값을 이용해서 접근

numbers [행의 index][열의 index]

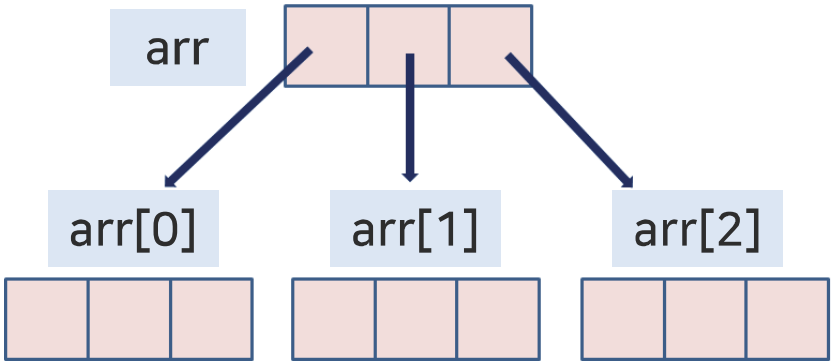
2차원 배열의 첫 번째 행의 첫 번째 요소에 아래와 같이 값을 대입

```
numbers [0][0] = 10;
```

2차원 배열의 접근

2차원 배열의 메모리구조

```
int[] [] arr = new int[3][3];
```



2차원 배열 arr은 1차원 배열의 주소 값을 가지는 1차원 배열

arr[0]은 2차원 배열이 가지는 첫 번째 1차원 배열, arr[1]은 두 번째, arr[2]는 세 번째 1차원 배열

첫 번째 1차원 배열의 첫 번째 요소는 arr[0][0]이 됨

2차원 배열의 접근

```
package chapter06;

public class ArrayTest4 {

	public static void main(String[] args) {

		// 2차원 배열 인스턴스 생성
		int[][] arr = new int[2][2];

		// 배열 요소에 값 대입
		arr[0][0] = 11;
		arr[0][1] = 12;
		arr[1][0] = 21;
		arr[1][1] = 22;
```

2차원 배열의 접근

```
            // 배열 요소 출력
            System.out.println(arr[0][0]);
            System.out.println(arr[0][1]);
            System.out.println(arr[1][0]);
            System.out.println(arr[1][1]);
        }
}
```

배열 선언과 동시에 요소 초기화하기

```
자료형타입[열의 길이] [행의 길이] 배열이름= {
    {배열 요소, 배열 요소,...},
    {배열 요소, 배열 요소,...},
    ...
};
```

배열 선언과 동시에 요소 초기화하기

```
package chapter06;

public class ArrayTest5 {

        public static void main(String[] args) {

                // 배열 인스턴스 생성과 초기화
                int[][] scores = {
                        { 100, 90, 80 }, { 98, 76, 90 }, { 85, 84, 77 }, { 89, 100, 69 }
                };
```

배열 선언과 동시에 요소 초기화하기

```
            // 2차원 배열의 참조
            System.out.println("scores[0][0] => " + scores[0][0]);
            System.out.println("scores[1][1] => " + scores[1][1]);
            System.out.println("scores[2][2] => " + scores[2][2]);
            System.out.println("scores[3][2] => " + scores[3][2]);

        }
}
```

배열의 순차적 접근

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Java
C++

배열의 순차적 접근

배열 인스턴스에는 요소의 개수를 저장하고 있는 length 필드를 가지고 있음

```
int[][] numbers = new int[4][3];

numbers.length;
```

2차원 배열의 행의 개수 : 4
2차원 배열이 가지고 있는 1차원 배열의 개수를 저장하고 있습니다.

```
numbers[0].length
```

2차원 배열의 첫 번째 행 배열의 개수 : 3
첫 번째 1차원 배열의 요소 개수를 저장하고 있습니다.

for 반복문을 이용한 배열처리

```
package chapter06;
public class ArrayTest6 {
        public static void main(String[] args) {
                int[][] scores = {  // 배열 인스턴스 생성과 초기화
                                {100, 90, 80},
                                {98, 76, 90},
                                {85, 84, 77},
                                {89, 100, 69}
                              };
                for(int i=0; i<scores.length; i++){ // 행의 반복 : 변수 i는 행의          index
                        for(int j=0; j<scores[i].length; j++){ // 열의 반복 : 변수 j는 열의 index
                                System.out.print("numbers["+i+"]["+j+"] => " + scores[i][j] + ", \t");
                        }
                        System.out.println();
                }
        }
}
```

향상된 for 문(The enhanced for statement)을 이용한 배열 처리

```
for ( 자료형타입 변수이름 : 배열 또는 컬렉션의 원본) {
// 배열이나 컬렉션의 요소 개수만큼 순차적으로 실행합니다.
   // 각 요소 처리할 문장을 기술합니다.
 }
```

for 다음에 오는 괄호 안의 첫 번째 변수는 배열의 각 요소를 저장하는 변수

배열을 반복하면서 각 요소 데이터를 이 변수에 저장

{} 블록 안에서 이 변수를 이용해서 데이터 처리

향상된 for 문(The enhanced for statement)을 이용한 배열 처리

```
package chapter06;

public class ArrayTest7 {

	public static void main(String[] args) {
		// 배열 인스턴스 생성
		int[] score1 = { 100, 90, 70, 80, 65 };

		System.out.println("배열 score1 =====");
		for (int i : score1) {
			System.out.println(i);
		}

	}
}
```

향상된 for 문(The enhanced for statement)을 이용한 배열 처리

```
package chapter06;

public class ArrayTest8 {

        public static void main(String[] args) {

                int[][] scores = { // 배열 인스턴스 생성과 초기화
                                { 100, 90, 80 }, { 98, 76, 90 } };

                for (int[] is : scores) { // 행의 반복

                        for (int i : is) { // 열의 반복
```

향상된 for 문(The enhanced for statement)을 이용한 배열 처리

```
                                System.out.print(i + ", \t"); // 변수 i에 배열의 각 요소 값이
저장되어 반복하면 출력
                        }
                        System.out.println();
                }

        }

}
```

객체를 다루는 배열

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Java
C++

객체를 다루는 배열

클래스에는 여러 가지 타입의 변수를 선언하여 객체를 만들 수 있음

이때 만들어지는 객체도 여러 가지 변수를 묶음으로 사용한다는 것

배열이 객체를 요소로 가질 수 있음 → 다차원 배열을 쓰는 것과 유사

객체를 다루는 배열

```
package chapter06;
public class Member {
        int memberNo;
        String memberId;
        String memberName;
        public Member(int memberNo, String memberId, String memberName) {
                this.memberNo = memberNo;
                this.memberId = memberId;
                this.memberName = memberName;
        }
        @Override
        public String toString() {
                return memberNo + ". " + memberId + " (" + memberName + ")";
        }
}
```

객체를 다루는 배열

```
package chapter06;

public class ArrayTest9 {

    public static void main(String[] args) {
        // Member클래스 타입의 배열 5개 생성 합니다.
        Member[] members = new Member[5];

        // 배열의 요소에 참조 값 대입 합니다.
        members[0] = new Member(1, "cool", "시원한");
        members[1] = new Member(2, "hot", "뜨거운");
        members[2] = new Member(3, "son", "손흥민");
        members[3] = new Member(4, "park", "박지성");
        members[4] = new Member(5, "cha", "차두리");
```

객체를 다루는 배열

```
        // 배열의 요소인 참조 변수를 통해 객체의 멤버에 참조합니다.
        for (int i = 0; i < members.length; i++) {
            System.out.printf("%d번 회원의 아이디는 %s 이고, 이름은 %s 입니다.\n",
members[i].memberNo, members[i].memberId,
                    members[i].memberName);
        }
    }
}
```